

1. Explain the Two-Phase Locking Protocol. How does it guarantee serializability? The Two-Phase Locking (2PL) Protocol is a concurrency control method that ensures serializability in database transactions. It operates in two distinct phases:

- **Growing Phase:** In this phase, a transaction can acquire locks but cannot release any locks. It continues to acquire all the locks it needs for its operations.
- **Shrinking Phase:** In this phase, a transaction can release locks but cannot acquire any new locks. Once a transaction releases its first lock, it enters the shrinking phase. **How it guarantees serializability:** 2PL guarantees serializability because it prevents conflicts that would lead to non-serializable schedules. By ensuring that a transaction holds all its required locks before it starts releasing any, it prevents other transactions from acquiring those locks and potentially causing inconsistencies. If two transactions follow 2PL, their interleaved execution will be equivalent to some serial execution of those transactions. A strict 2PL, where exclusive locks are held until commit/rollback, further guarantees recoverability and cascade-less schedules.

2. What is deadlock and how can it be prevented? Describe common deadlock-prevention schemes.

Deadlock is a situation in a multi-transaction environment where two or more transactions are indefinitely waiting for each other to release resources that they need. This results in a standstill, as none of the transactions can proceed. For example, transaction A holds resource X and needs resource Y, while transaction B holds resource Y and needs resource X. **Deadlock Prevention Schemes:** These schemes ensure that a deadlock never occurs by restricting how requests for resources can be made. Common prevention schemes include:

- **Pre-allocation/Ordering of Resources:** Transactions must request all the resources they will need before they begin execution. Alternatively, resources can be ordered globally, and transactions must request resources in increasing order of enumeration. This prevents circular waiting.
- **Wait-Die Scheme:** If a transaction T_i requests a resource held by T_j :
 - If T_i is older than T_j (has an earlier timestamp), T_i is allowed to wait for T_j (Wait).
 - If T_i is younger than T_j , T_i is aborted and restarted with a random delay (Die). This avoids T_i waiting for T_j if T_j is going to finish soon.
- **Wound-Wait Scheme:** If a transaction T_i requests a resource held by T_j :
 - If T_i is older than T_j , T_j is aborted and restarted (Wound), freeing the resource for T_i .
 - If T_i is younger than T_j , T_i waits for T_j (Wait). This prioritizes older transactions. Both Wait-Die and Wound-Wait use timestamps to enforce a total ordering of transactions, preventing cycles in the wait-for graph.

3. Compare primary, secondary and clustering indexes. Which of these indexes are dense and which are not? How is implementation of clustering indexes performed?

- **Primary Index:** A primary index is an index on a field that is the primary key of the table. It is always a sparse index (non-dense) if the file is ordered sequentially by the primary key,

containing one entry for each block in the data file. Each entry points to the first record in that block. It guarantees unique values and ordered data access.

- **Secondary Index:** A secondary index is an index on a non-primary key field, which may not be unique. It is always a dense index, containing an entry for every record in the data file. Each entry contains the search key value and a pointer to the record (or a bucket of pointers if the search key is not unique). Secondary indexes are used for efficient retrieval based on non-primary key attributes.
- **Clustering Index:** A clustering index is an index on a non-key field that may have duplicate values. The data file records themselves are ordered physically based on the clustering field. There can be at most one clustering index per table.
 - **Dense vs. Non-Dense:** Primary indexes are typically non-dense (sparse). Secondary indexes are always dense. Clustering indexes are typically non-dense (sparse), with one entry per distinct value of the clustering key, pointing to the first record with that value.
 - **Implementation of Clustering Indexes:** When a clustering index is created, the database system physically reorders the records in the table on disk according to the values of the clustering key. The index then stores an entry for each distinct value of the clustering field, pointing to the block containing the first record with that value. Subsequent records with the same clustering key value are stored contiguously. This physical ordering allows for very efficient retrieval of records with the same clustering key value, as they are located together on disk, minimizing disk I/O.

4. How do B-tree indexes differ from binary search tree indexes?

- **Binary Search Tree (BST) Indexes:**
 - Each node has at most two children (left and right).
 - Values in the left subtree are less than the node's value, and values in the right subtree are greater.
 - Designed for in-memory data structures.
 - Can become unbalanced (skewed) if data is inserted or deleted in a particular order, leading to a worst-case $O(N)$ search time, similar to a linked list.
 - Poor performance for disk-based databases due to high disk I/O caused by potentially many random reads (one for each node traversed).
- **B-tree Indexes:**
 - Are self-balancing search trees specifically designed for disk-based storage.
 - Nodes can have many children (typically dozens to hundreds), determined by the "order" of the B-tree. This wide branching factor reduces the height of the tree significantly compared to a BST for the same amount of data.
 - All leaf nodes are at the same depth.

- Minimize disk I/O operations by maximizing the amount of data stored in each node (which typically corresponds to a disk block). A search or update operation involves traversing only a few nodes (disk reads).
- Guaranteed logarithmic time complexity for search, insertion, and deletion operations ($O(\log_b N)$, where b is the branching factor). This makes them highly efficient for large databases.
- Internal nodes store keys and pointers to child nodes, while leaf nodes typically store keys and pointers to actual data records or blocks.

5. What is a B+ -tree? Why is a B+ -tree better structure than a B-Tree for implementation of an index sequential file? A **B+ -tree** is a variant of the B-tree that is widely used for indexing in databases and file systems. The key differences from a B-tree are:

- **Data Storage:** All data pointers (pointers to actual records or data blocks) are stored only at the leaf nodes. Internal nodes contain only key values and pointers to child nodes, acting solely as navigational guides.
- **Leaf Node Linking:** All leaf nodes are linked together in a sequential manner (typically a doubly linked list). This allows for efficient range queries.

Why is a B+ -tree better than a B-tree for implementation of an index sequential file?

- **Efficient Range Queries:** The linked list of leaf nodes in a B+ -tree allows for very efficient range queries (e.g., "find all records where salary is between 50,000 and 70,000"). Once the start of the range is found in a leaf node, subsequent records within the range can be retrieved by simply traversing the linked list, avoiding the need to ascend and descend the tree multiple times as might be necessary in a standard B-tree. This sequential access is crucial for index sequential files.
- **Optimized Disk I/O for Internal Nodes:** Because internal nodes only store keys and child pointers (no data pointers), more keys can fit into a single disk block. This increases the branching factor, leading to a shallower tree and fewer disk I/O operations required to traverse the tree to find a leaf node.
- **Uniformity in Search:** Every search in a B+ -tree, whether for a specific record or a range, must traverse from the root to a leaf node. This predictability in access path can be beneficial for performance tuning.
- **Better Caching:** The separation of index and data pointers allows for more effective caching. Internal index nodes can be cached in memory without also caching data blocks, improving overall cache utilization.

6. Write SQL commands for each of the following: (i) **Creation of views:** `sql CREATE VIEW HighPaidEmployees AS SELECT EmpID, EmpName, Salary FROM Employees WHERE Salary > 50000;` (ii) **Creation of sequences:** `sql CREATE SEQUENCE EmployeeID_Seq START WITH 1 INCREMENT BY 1 MINVALUE 1 MAXVALUE 999999 CACHE 20;` (iii) **Outer join:** ```sql -- LEFT OUTER JOIN SELECT`

```
E.EmpName, D.DeptName FROM Employees E LEFT OUTER JOIN Departments D ON E.DeptID = D.DeptID;
```

```
-- RIGHT OUTER JOIN
```

```
SELECT E.EmpName, D.DeptName
```

```
FROM Employees E
```

```
RIGHT OUTER JOIN Departments D ON E.DeptID = D.DeptID;
```

```
-- FULL OUTER JOIN (not supported by all DBMS, use UNION of LEFT and RIGHT if not)
```

```
SELECT E.EmpName, D.DeptName
```

```
FROM Employees E
```

```
FULL OUTER JOIN Departments D ON E.DeptID = D.DeptID;
```

```
...
```

(iv) **To give access permission to any user:** sql GRANT SELECT, INSERT ON Employees TO UserName; --
To grant all privileges on a table to a user GRANT ALL PRIVILEGES ON Employees TO UserName; -- To
grant a role to a user GRANT RoleName TO UserName;

7. Why is data replication useful in Distributed Database Management System (DDBMS)? Briefly discuss the terms 'Complete Replication' and 'Selective Replication'. Usefulness of Data Replication in DDBMS: Data replication involves storing multiple copies of the same data at different sites in a distributed database system. It is useful for several reasons:

- **Increased Availability:** If one site fails, copies of the data are still available at other sites, ensuring continuous operation and high availability.
- **Improved Reliability:** Redundancy of data protects against data loss due to hardware failures or disasters at a single site.
- **Faster Query Response:** Queries can be processed at the site closest to the user or the site with the least load, reducing network traffic and improving response times, especially for read-intensive operations.
- **Load Balancing:** Workload can be distributed among multiple sites holding copies of the data.
- **Reduced Network Traffic:** Local access to data reduces the need to access remote sites, minimizing network communication.

Types of Replication:

- **Complete Replication:** In this scheme, a copy of the *entire* database is stored at each site in the DDBMS.

- **Advantages:** Maximizes availability and query performance as any query can be answered locally at any site. High fault tolerance.
- **Disadvantages:** High storage overhead due to multiple full copies. Significant overhead for updates, as every update must be propagated to all copies across all sites, increasing network traffic and complexity for concurrency control (maintaining consistency across all replicas).
- **Selective Replication (Partial Replication):** In this scheme, only specific fragments or portions of the database that are frequently accessed or critical are replicated across selected sites. Not all data is replicated, and not all sites necessarily have the same replicated data.
 - **Advantages:** Reduces storage overhead compared to complete replication. Less update traffic and simpler consistency management compared to complete replication, as updates only need to be propagated to relevant sites.
 - **Disadvantages:** Availability and query performance might be lower than complete replication for data that is not replicated or not replicated locally. Query processing might require access to remote sites for non-replicated data.

8. Explain the term data replication and data fragmentation with suitable example.

- **Data Replication:** Data replication is the process of storing multiple copies of the same data at different locations (sites) within a distributed database system. The primary goal is to enhance data availability, reliability, and query performance.
 - **Example:** Consider an online retail company with branches in New York, London, and Tokyo. To ensure fast access to product information and high availability, the Products table, which contains details of all products, could be replicated entirely at all three locations. If the New York server goes down, customers in London and Tokyo can still view and order products from their local database copies. Any update to a product (e.g., price change) would need to be synchronized across all three replicated copies.
- **Data Fragmentation:** Data fragmentation is the process of dividing a single logical database into smaller, distinct logical units called fragments. These fragments can then be stored at different sites in a distributed database system. Fragmentation aims to improve query locality, concurrency, and security.
 - **Types of Fragmentation:**
 - **Horizontal Fragmentation:** Divides a relation into subsets of tuples (rows) based on a condition. Each fragment contains a subset of the original rows.
 - **Example:** A Customers table with customers from different regions might be horizontally fragmented. Customers_NY (customers from New York) and Customers_LA (customers from Los Angeles) could be stored at their respective regional offices.

- **Vertical Fragmentation:** Divides a relation into subsets of attributes (columns). Each fragment contains a subset of the original columns for all tuples, and a common key column to allow reconstruction.
 - **Example:** An Employees table with columns like EmpID, Name, Address, Salary, Department, and PhoneNumber could be vertically fragmented. Emp_HR (containing EmpID, Name, Salary, Department) could be stored at the HR department's site, while Emp_Contact (containing EmpID, Name, Address, PhoneNumber) could be stored at a general administration site. Both fragments would share EmpID and Name to facilitate joining them back.
- **Mixed/Hybrid Fragmentation:** A combination of horizontal and vertical fragmentation.

9. What are integrity constraints? Explain the various types of integrity constraints. Integrity Constraints are rules that define what data values are permissible in the database and which relationships among values must be maintained. They are used to ensure the accuracy, consistency, and reliability of the data within a relational database. When data is inserted, updated, or deleted, the database management system (DBMS) checks these constraints to ensure data validity. **Various types of integrity constraints:**

- **Domain Constraints:** These specify that all values in a column must be drawn from a predefined domain (set of allowed values). This includes data types (e.g., INTEGER, VARCHAR, DATE) and specific ranges or formats.
 - **Example:** A column Age must be an INTEGER and values must be between 0 and 120. A Gender column can only accept 'M' or 'F'.
- **Entity Integrity Constraint:** This states that the primary key of a table cannot contain NULL values. This is because the primary key uniquely identifies each tuple (row), and a NULL value would mean that a tuple cannot be uniquely identified.
 - **Example:** In an Employees table, the EmpID (primary key) cannot be NULL.
- **Referential Integrity Constraint (Foreign Key Constraint):** This constraint ensures that a value in one table that refers to a primary key in another table (a foreign key) must either be NULL or exist as a valid primary key value in the referenced table. It maintains consistency between related tables.
 - **Example:** In a Departments table (DeptID primary key) and an Employees table (DeptID foreign key referencing Departments), every DeptID in the Employees table must either correspond to an existing DeptID in the Departments table or be NULL (if an employee can temporarily not be assigned to a department).
- **Key Constraints (Unique Constraints):** These constraints specify that the values in a particular column or a set of columns must be unique for all tuples in the table. Unlike a primary key, a unique key can allow NULL values (though only one NULL, depending on the DBMS).

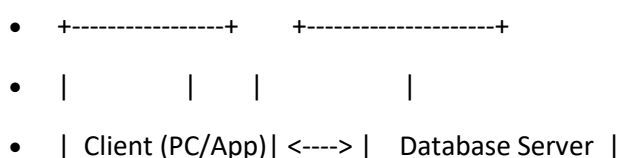
- **Example:** An Email column in a Users table might have a unique constraint to ensure no two users have the same email address.
- **Check Constraints:** These allow defining a condition that each row in the table must satisfy. The condition is a boolean expression that evaluates to true or false.
 - **Example:** A Salary column in an Employees table must be greater than 0 (CHECK (Salary > 0)). A Quantity column in an OrderDetails table must be positive (CHECK (Quantity > 0)).

10. Describe the following client-server architectures with the help of a diagram: (i) 2-tier (ii) 3-tier (As I cannot create diagrams, I will describe them in detail.) **Client-Server Architecture:** A distributed computing framework where the overall task is divided between service providers (servers) and service requesters (clients).

(i) 2-Tier Architecture:

- **Description:** In a 2-tier architecture, the system is logically divided into two main components: the client (front-end) and the server (back-end).
 - **Client (Presentation Layer + Application Logic):** The client typically handles the user interface (presentation layer) and a significant portion of the application logic (business logic). This can include data validation, query generation, and some business rules.
 - **Server (Data Layer):** The server primarily consists of the database management system (DBMS). Its main role is to store, manage, and provide access to the data. It receives requests from the client, processes them (executes SQL queries), and sends the results back to the client.
- **Communication:** Direct communication between the client and the database server.
- **Advantages:** Relatively simple to develop and deploy for small to medium-sized applications. Good performance for a limited number of users because of direct connection.
- **Disadvantages:**
 - **Scalability Issues:** As the number of clients increases, the server can become overloaded due to numerous direct connections and the processing load of application logic.
 - **Maintenance:** Changes to application logic often require updating all client installations.
 - **Security:** Direct exposure of the database to clients can raise security concerns.
 - **Limited Reusability:** Application logic embedded in clients is difficult to reuse.

• **Conceptual Diagram:**

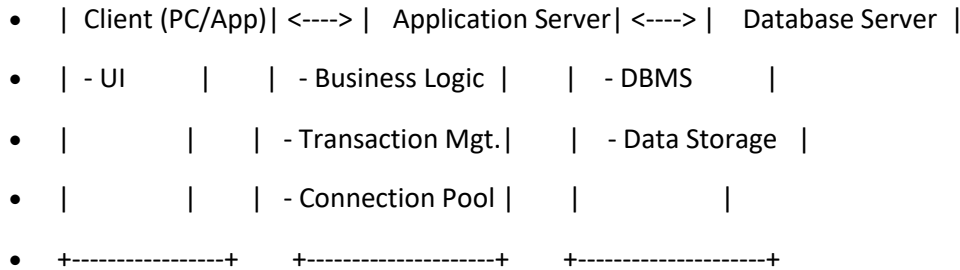


- | - UI | | - DBMS |
- | - App Logic | | - Data Storage |
- | | |
- +-----+ +-----+

(ii) 3-Tier Architecture:

- **Description:** A 3-tier architecture introduces an intermediate layer between the client and the database server, typically called the "Application Server" or "Middleware." This separates the application logic from both the presentation layer and the data layer.
 - **Client (Presentation Layer):** The client layer (thin client) focuses solely on the user interface and presentation. It sends requests to the application server and displays the results. Little to no business logic resides here.
 - **Application Server (Logic Layer):** This is the crucial middle tier. It contains the majority of the application logic, business rules, and transaction processing. It receives requests from clients, processes them, and interacts with the database server on behalf of the clients. It can also manage connection pooling, security, and load balancing.
 - **Database Server (Data Layer):** Similar to the 2-tier model, this layer stores and manages the data. It responds to requests from the application server.
- **Communication:** Client communicates with the Application Server, and the Application Server communicates with the Database Server.
- **Advantages:**
 - **Improved Scalability:** The application server can manage connections efficiently and distribute load, supporting a larger number of users. It can be scaled independently.
 - **Enhanced Maintainability:** Business logic resides centrally on the application server, making updates and maintenance easier.
 - **Better Security:** The database is not directly exposed to clients; all interactions go through the controlled application server.
 - **Increased Reusability:** Business logic components on the application server can be reused by various clients or even other applications.
 - **Flexibility:** Different client types (web, desktop, mobile) can interact with the same application server.
- **Disadvantages:** More complex to design, develop, and deploy due to the additional layer.
- **Conceptual Diagram:**

- +-----+ +-----+ +-----+
- | | | | |



11. What is a hashed file organization and inverted file organisation? What are its advantages and disadvantages?

- **Hashed File Organization:**
 - **Description:** In hashed file organization, the physical location of a record on disk is determined by a hash function applied to a key field (hash key) of the record. The hash function computes a bucket address (or block address) where the record is likely to be stored. If multiple records hash to the same bucket (collision), they are stored in the same bucket or in overflow areas linked from that bucket.
 - **Advantages:**
 - **Very Fast Direct Access:** Provides extremely fast direct access to records when the hash key is known, often in a single disk I/O for successful lookups, making it ideal for exact-match queries.
 - **Efficient for Frequent Exact Searches:** Suitable for applications where specific records need to be retrieved quickly.
 - **Disadvantages:**
 - **Poor for Range Queries:** Inefficient for range queries or sequential access, as records are scattered across the disk based on the hash function, not on their logical order.
 - **Collision Handling Overhead:** Performance can degrade significantly if the hash function produces too many collisions, leading to long chains in overflow areas and increased disk I/O.
 - **Space Utilization:** Can lead to wasted space if the hash table is not properly sized or if dynamic hashing is not used.
 - **Hash Function Dependence:** Performance is heavily dependent on the quality of the hash function; a poor hash function can lead to uneven distribution and high collision rates.
- **Inverted File Organization (Inverted Index):**
 - **Description:** An inverted file organization (more commonly known as an inverted index) is a data structure used to map content (e.g., words, values in a field) to their locations

in a document or database. It essentially creates an index for every "searchable" word or value. It consists of two main parts:

- **Vocabulary/Lexicon:** A sorted list of all unique search terms (words, values).
- **Posting List:** For each term in the vocabulary, a list of pointers (or document IDs, record IDs, block addresses) indicating where that term appears in the data.
- **Advantages:**
 - **Efficient for Keyword/Multi-attribute Searches:** Excellent for full-text search systems (like search engines) and complex queries involving multiple attributes (e.g., "find all employees who are in the 'Sales' department AND have 'Project Manager' as a designation").
 - **Faster Retrieval for Non-key Attributes:** Can quickly locate records based on values in any indexed attribute, even if it's not a primary key.
 - **Flexible Querying:** Supports various types of queries, including boolean operations (AND, OR, NOT) on terms.
- **Disadvantages:**
 - **High Storage Overhead:** Can require significant storage space, especially if many attributes are indexed or if the data contains a large vocabulary (e.g., text documents).
 - **Update Complexity:** Updates (insertions, deletions, modifications of records) can be expensive as they require updating multiple posting lists in the index.
 - **Slower for Sequential Scan:** Not efficient for simply scanning the entire file or for range queries on non-indexed attributes.

12. Explain the meaning of the following two keywords: Commit and Rollback. Write a code fragment for transferring money from account A to account B and show the uses of Commit and Rollback. In the context of database transactions, **Commit** and **Rollback** are crucial commands for ensuring data integrity and atomicity.

- **Commit:** The COMMIT command is used to permanently save the changes made during the current transaction to the database. Once a COMMIT is executed, all operations within that transaction are considered successful and durable. The changes become visible to other transactions, and the database reaches a new consistent state.
- **Rollback:** The ROLLBACK command is used to undo all the changes made during the current transaction since the last COMMIT or SAVEPOINT. If a transaction encounters an error, or if the user decides to cancel the operations, ROLLBACK restores the database to its state before the transaction began, ensuring atomicity (all or nothing).

Code Fragment for Transferring Money:

Let's assume we have a table Accounts with columns AccountID and Balance.

SQL

-- Start a new transaction

START TRANSACTION; -- or BEGIN TRANSACTION; or simply start operations depending on DBMS default

-- Step 1: Deduct money from Account A

UPDATE Accounts

SET Balance = Balance - 100.00 -- Assuming transferring 100.00

WHERE AccountID = 'A';

-- Check if deduction was successful (e.g., balance didn't go negative or A exists)

-- This is a simplified check; in a real scenario, more robust error handling and checks are needed.

-- For demonstration, let's assume a trigger or constraint ensures positive balance.

-- Step 2: Add money to Account B

UPDATE Accounts

SET Balance = Balance + 100.00

WHERE AccountID = 'B';

-- Check if addition was successful (e.g., B exists)

-- Decision Point:

-- If both updates were successful and no errors occurred:

-- Permanently save the changes

COMMIT;

-- ELSE (If any error occurred during the process, or if Account A didn't have enough balance, etc.):

-- Revert all changes made during this transaction

-- For example, if Account 'A' did not exist or insufficient funds:

-- ROLLBACK;

Explanation of Use: In this money transfer example, the two UPDATE statements constitute a single logical transaction.

- If both UPDATE operations execute successfully, we issue a COMMIT. This ensures that the money is deducted from Account A AND added to Account B. Both operations are made permanent.
- If, however, the first UPDATE fails (e.g., AccountID 'A' doesn't exist, or Balance - 100.00 would result in a negative balance and a check constraint is violated), or if the second UPDATE fails (e.g., AccountID 'B' doesn't exist), then we would issue a ROLLBACK. This would undo the first UPDATE (if it had occurred) and ensure that Account A's balance is restored to its original state, preventing a scenario where money is deducted from A but not added to B (or vice-versa). This guarantees the atomicity of the money transfer.

13. What are Checkpoints? How does this technique of Checkpoints contribute to database recovery?

Give suitable example. Checkpoints are a technique used in database management systems to reduce the amount of work needed during database recovery after a system crash. A checkpoint is a point in time when the DBMS ensures that all log records corresponding to transactions that have committed up to that point have been written to stable storage (disk), and all modified data pages from those committed transactions have also been written from the buffer cache to disk. **How Checkpoints**

Contribute to Database Recovery: During recovery after a crash, the DBMS needs to bring the database back to a consistent state. This typically involves:

- **Redo operations:** Reapplying changes of committed transactions that were not written to disk before the crash.
- **Undo operations:** Undoing changes of uncommitted transactions (or partially completed transactions) that were written to disk before the crash. Without checkpoints, the DBMS would have to scan the entire transaction log from the beginning of time to identify all committed and uncommitted transactions, which can be a very long and time-consuming process for large logs. Checkpoints significantly reduce the recovery time and effort by:
- **Limiting Log Scan:** When a system recovers from a crash, it only needs to process the transaction log from the last successful checkpoint, rather than the entire log from the database's inception. All transactions committed *before* the checkpoint are guaranteed to have their changes on disk, so they don't need to be redone.
- **Identifying Active Transactions:** Checkpoints record information about transactions active at the time of the checkpoint, helping the recovery manager quickly identify which transactions need to be undone (those active but not committed at the crash time) and which might need to be redone (those committed after the checkpoint but before the crash).

Suitable Example: Imagine a database for an online store.

- **Scenario without Checkpoints:** If the system crashes, the recovery process would need to examine every single transaction ever recorded in the log to determine which ones committed and which didn't, and what data changes need to be applied or rolled back. This could take hours.

- **Scenario with Checkpoints:**

1. At 10:00 AM, the system takes a **checkpoint**. All completed transactions and their data changes up to 10:00 AM are guaranteed to be on disk.
2. From 10:00 AM to 10:30 AM, several new transactions (T1, T2, T3, T4, T5) start.
3. At 10:20 AM, T1 and T2 commit. Their changes are written to the log and potentially to disk.
4. At 10:25 AM, T3 commits.
5. At 10:30 AM, the system crashes. T4 and T5 were still active and uncommitted.

Recovery using Checkpoint:

- The recovery process starts by finding the last checkpoint (10:00 AM).
- It then scans the log *only from 10:00 AM onwards* (a much smaller segment).
- It identifies that T1, T2, and T3 committed between 10:00 AM and 10:30 AM. It will ensure their changes are redone if they weren't fully written to disk.
- It identifies that T4 and T5 were active but uncommitted at 10:30 AM. Their partial changes (if any were written to disk) will be undone. This significantly reduces the recovery time compared to scanning the entire log.

14. What is a data dictionary? What should be included in data dictionary? A data dictionary (also known as metadata repository or system catalog) is a centralized repository of information about data, such as meaning, relationships to other data, origin, usage, and format. It essentially stores "data about data" or metadata. It's a critical component of any DBMS, acting as a knowledge base for the database system itself and for users. **What should be included in a data dictionary:** A comprehensive data dictionary typically includes information about:

- **Tables/Relations:**

- Table names, creation dates, last modified dates.
- Owner of the table.
- Storage details (e.g., size, location on disk).

- **Columns/Attributes:**

- Column names, data types (e.g., VARCHAR, INTEGER, DATE), lengths.
- Nullability (whether NULL values are allowed).
- Default values.
- Constraints associated with the column (e.g., UNIQUE, CHECK constraints).

- **Indexes:**

- Index names, associated tables and columns.

- Type of index (e.g., primary, secondary, clustering, B-tree).
- Whether it's unique or not.
- **Views:**
 - View names and their definitions (the SQL query that defines the view).
 - Base tables involved in the view.
- **Constraints:**
 - Definitions of all integrity constraints (primary key, foreign key, unique, check constraints), including the tables and columns they apply to.
 - Referential integrity actions (e.g., ON DELETE CASCADE, ON UPDATE NO ACTION).
- **Users and Permissions (Security Information):**
 - Usernames, roles, and groups.
 - Privileges granted to users/roles on specific database objects (tables, views, procedures).
- **Stored Procedures, Functions, Triggers:**
 - Names, definitions, parameters, and associated tables for these programmable objects.
- **Synonyms:**
 - Names and their corresponding object mappings.
- **Database Schema Information:**
 - Overall schema structure and relationships between tables.
- **Statistical Information:**
 - Statistics about data distribution, number of rows in tables, number of distinct values in columns. This information is crucial for the query optimizer.

15. What is Optimization in DBMS? How does relational algebra help in Optimizing the Query? Discuss with suitable example. List the operators used in relational algebra and briefly explain them.

Optimization in DBMS (Query Optimization): Query optimization is the process of choosing the most efficient execution plan for a given SQL query. When a user submits an SQL query, there are often many different ways the DBMS can execute it (e.g., different join orders, different index usages). The query optimizer's goal is to select the plan that minimizes the query's cost, usually measured in terms of I/O operations and CPU time. This is crucial for database performance, especially for complex queries on large datasets.

How Relational Algebra Helps in Optimizing the Query: Relational algebra is a procedural query language that defines the theoretical operations on relations (tables). Before an SQL query is executed, it is often translated into an equivalent relational algebra expression. This is where optimization begins:

1. **Logical Query Optimization:** The relational algebra expression can be transformed and rewritten into equivalent, but potentially more efficient, expressions. The optimizer applies algebraic equivalences (rules for transforming relational algebra expressions) to create various alternative execution plans. For example, applying selection (filter) operations as early as possible (before joins) can significantly reduce the number of rows that need to be processed in subsequent operations.
2. **Physical Query Optimization:** After generating different logical plans, the optimizer considers physical access paths (e.g., presence of indexes, available join algorithms like nested loop join, hash join, sort-merge join) and statistical information (from the data dictionary) to estimate the cost of each logical plan. It then chooses the plan with the lowest estimated cost.

Suitable Example: Consider two tables: Customers (CustomerID, Name, City) and Orders (OrderID, CustomerID, OrderDate, Amount). **SQL Query:**

SQL

SELECT C.Name, O.OrderDate

FROM Customers C, Orders O

WHERE C.CustomerID = O.CustomerID AND C.City = 'London' AND O.Amount > 1000;

Possible Relational Algebra Execution Plans (Simplified):

Plan 1 (Less Optimized):

1. Join Customers and Orders on CustomerID. $R_1 \leftarrow \text{Customers} \bowtie_{\text{CustomerID}} \text{Orders}$
2. Select (filter) from R_1 where City = 'London' AND Amount > 1000.
 $R_2 \leftarrow \sigma_{\text{City}='London' \wedge \text{Amount} > 1000}(R_1)$
3. Project (select columns) Name and OrderDate from R_2 .
 $\text{Result} \leftarrow \pi_{\text{Name}, \text{OrderDate}}(R_2)$
 - **Issue:** The join operation Customers and Orders might produce a very large intermediate table (R_1) before the filtering happens, consuming significant resources.

Plan 2 (More Optimized using Relational Algebra rules - Pushdown Selection):

1. Select from Customers where City = 'London'. $R_C \leftarrow \sigma_{\text{City}='London'}(\text{Customers})$
2. Select from Orders where Amount > 1000. $R_O \leftarrow \sigma_{\text{Amount} > 1000}(\text{Orders})$
3. Join R_C and R_O on CustomerID. $R_J \leftarrow R_C \bowtie_{\text{CustomerID}} R_O$
4. Project Name and OrderDate from R_J . $\text{Result} \leftarrow \pi_{\text{Name}, \text{OrderDate}}(R_J)$
 - **Benefit:** By applying the selections (City = 'London' and Amount > 1000) *before* the join, the sizes of the relations being joined (R_C and R_O) are significantly reduced, leading to a much faster and less resource-intensive join operation. This is a classic example of "pushing down" selections.

Operators used in Relational Algebra:

- **Selection (σ):** Selects a subset of tuples from a relation that satisfy a given predicate (condition).
 - Example: $\sigma_{\text{Salary} > 50000}(\text{Employees})$
- **Projection (π):** Selects a subset of attributes (columns) from a relation. It also removes duplicate tuples from the result.
 - Example: $\pi_{\text{Name, Department}}(\text{Employees})$
- **Union ($\cup$):** Combines two relations (must be union-compatible, i.e., same number and types of attributes) by including all tuples from both, removing duplicates.
 - Example: $R \cup S$
- **Set Difference ($-$):** Returns tuples that are in the first relation but not in the second (must be union-compatible).
 - Example: $R - S$
- **Cartesian Product ($\times$):** Combines every tuple of the first relation with every tuple of the second relation. The resulting relation has attributes from both relations.
 - Example: $\text{Customer} \times \text{Orders}$ (often used as a precursor to join)
- **Rename (ρ):** Renames a relation or an attribute of a relation.
 - Example: $\rho_{\text{NewName}}(\text{OldRelation})$
- **Join (bowtie):** Combines tuples from two relations based on a join condition. Various types include:
 - **Natural Join (bowtie with common attribute):** Joins relations on all common attributes with the same name, removing duplicate common attributes.
 - **Theta Join (bowtie θ):** Joins relations based on an arbitrary join condition θ .
 - **Equijoin (bowtie $=$):** A specific type of theta join where the condition is equality.
- **Division ($\div$):** Used for queries that involve "for all" conditions (e.g., "find employees who work in all departments"). It is derived from other operations.
- **Assignment ($\leftarrow$):** Assigns the result of a relational algebra expression to a temporary relation.

16. Explain log-based recovery scheme and shadow paging recovery scheme with the help of an example. Log-Based Recovery Scheme:

- **Description:** This is the most widely used recovery scheme. It relies on a **transaction log** (or journal) which is a sequential record of all database modifications. Before any change is made to the actual database, an entry is written to the log. Each log record typically includes:

- Transaction ID.
- Type of operation (insert, delete, update).
- Table and record affected.
- Old value (before image) and new value (after image) for updates.
- Transaction status (start, commit, abort).
- **Recovery Process:**
 - **Undo:** If a transaction fails or is aborted, the log is used to restore the database to its state before that transaction began, by using the "before images" for updates or removing inserted records.
 - **Redo:** If the system crashes, the log is used to reapply the changes of all committed transactions whose changes might not have been written to disk before the crash, using "after images" for updates or re-inserting records.
- **Example:** Imagine a transaction T1 wants to update Balance for AccountID = 'A' from 500 to 400.
 1. **T1 starts.** Log record: <T1, START>
 2. **T1 updates account A.** Before writing the change to disk: Log record: <T1, UPDATE, AccountID='A', OldBalance=500, NewBalance=400> (Then the change is made to the data page in buffer, and eventually written to disk).
 3. **T1 commits.** Log record: <T1, COMMIT> **Scenario 1: Crash before commit record is written to log/disk:**
 - System crashes. During recovery, the log scan finds <T1, UPDATE, ...> but no <T1, COMMIT>.
 - **Result:** T1 is considered uncommitted. The change to Account A (from 500 to 400) is **undone** using OldBalance=500, restoring A's balance to 500. **Scenario 2: Crash after commit record is written to log, but data page not yet to disk:**
 - System crashes. During recovery, the log scan finds <T1, UPDATE, ...> AND <T1, COMMIT>.
 - **Result:** T1 is considered committed. The change to Account A is **redone** using NewBalance=400, ensuring A's balance is 400. This scheme ensures atomicity and durability.

Shadow Paging Recovery Scheme:

- **Description:** Shadow paging is a "write-ahead" recovery technique that avoids in-place updates of database pages. Instead, when a transaction modifies a page, a new copy of that page (a "shadow page") is created. All modifications are made to this shadow page. The original page remains untouched and represents the consistent state before the transaction.

- The database consists of a current page table (CPT) and a shadow page table (SPT). The CPT points to the current version of all database pages. The SPT points to the consistent state of the database *before* the current transaction(s) began.
- When a transaction makes a change, new physical pages are allocated, and modified data is written to these new pages. The CPT is updated to point to these new pages, while the SPT still points to the old, unmodified pages.
- **Recovery Process:**
 - **Commit:** To commit a transaction, the CPT is atomically switched to become the new SPT (i.e., the disk pointer to the root of the CPT is updated to point to the newly updated CPT). This makes the new state permanent. The old SPT (which was the CPT before the switch) and its associated old pages become garbage and can be deallocated.
 - **Rollback/Crash:** If a transaction aborts or the system crashes, no undo is needed. The system simply reverts to the previous consistent state by using the shadow page table (the SPT from before the transaction started). All the new, modified pages created by the failed transaction are simply discarded (garbage collected).
- **Example:** Imagine a database with two pages: Page 1 (Accounts A, B) and Page 2 (Accounts C, D).
 1. **Initial State:** CPT and SPT both point to the current versions of Page 1 and Page 2.
 2. **Transaction T1 starts:** T1 wants to update Account B on Page 1 and Account C on Page 2.
 3. **Modify Account B (on Page 1):**
 - A new shadow copy of Page 1, let's call it Page 1', is created.
 - The update to Account B is made on Page 1'.
 - The CPT is updated to point to Page 1' (for Page 1's entry). SPT still points to original Page 1.
 4. **Modify Account C (on Page 2):**
 - A new shadow copy of Page 2, Page 2', is created.
 - The update to Account C is made on Page 2'.
 - The CPT is updated to point to Page 2' (for Page 2's entry). SPT still points to original Page 2.
 5. **Commit T1:** The disk block containing the CPT is atomically updated to reflect the new state (pointing to Page 1' and Page 2'). The old Page 1 and Page 2 (pointed to by the old SPT) are now free. **Scenario 1: Transaction Aborts or System Crashes before Commit:**
 - The system simply discards the (partially) updated CPT and its new pages (Page 1', Page 2').

- The database is restored to the consistent state pointed to by the SPT (original Page 1 and Page 2). **Scenario 2: Transaction Commits successfully:**
- The new CPT becomes the definitive state. The old pages are released. **Advantages:** Simple crash recovery (no undo/redo analysis of log required). **Disadvantages:** High overhead due to copying entire pages. Data fragmentation can increase over time. Not suitable for concurrent transactions without additional mechanisms. Not as widely used as log-based recovery in commercial DBMS.